3 operations available on a single character

replace
delete
insert

Ex:

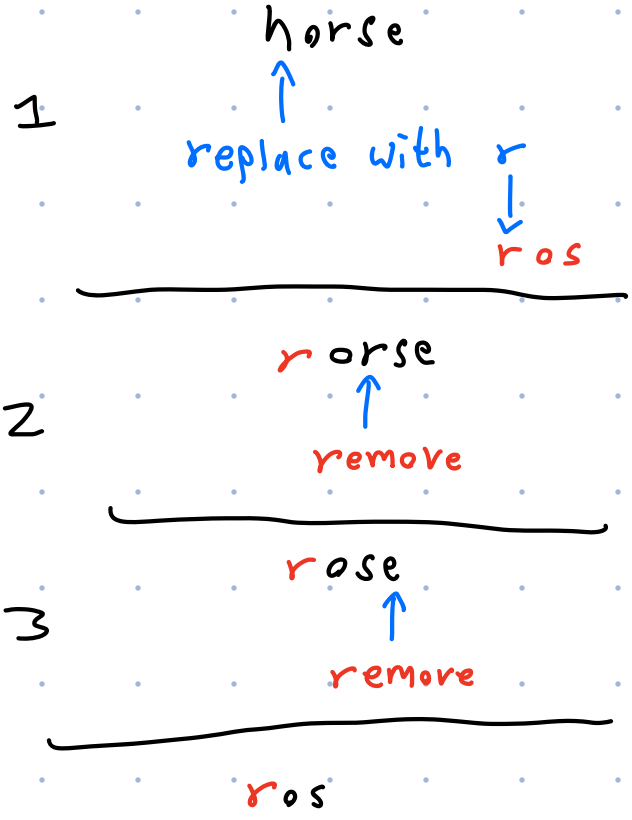
A = "horse"

B = "ros"

Transform A to B

minimum number of operations to change A to B is 3

minimum distance



"nicole"
0 1 2 3 4 5

(-1, 0)	""
(0, 0)	"n"
(0, 1)	"ni"
(0, 2)	"nic"
(0, 3)	"nico"
(0, 4)	"nicol"
(0, 5)	"nicole"

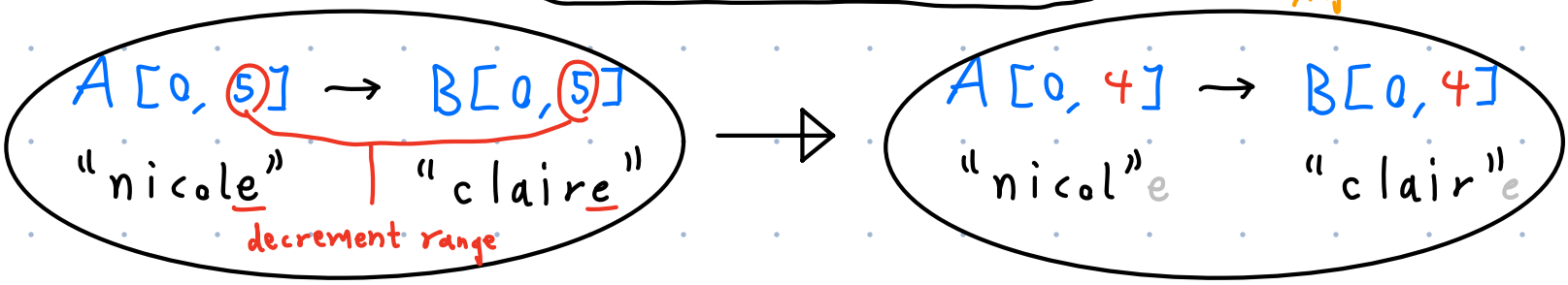
base case

A = "nicole"
0 1 2 3 4 5

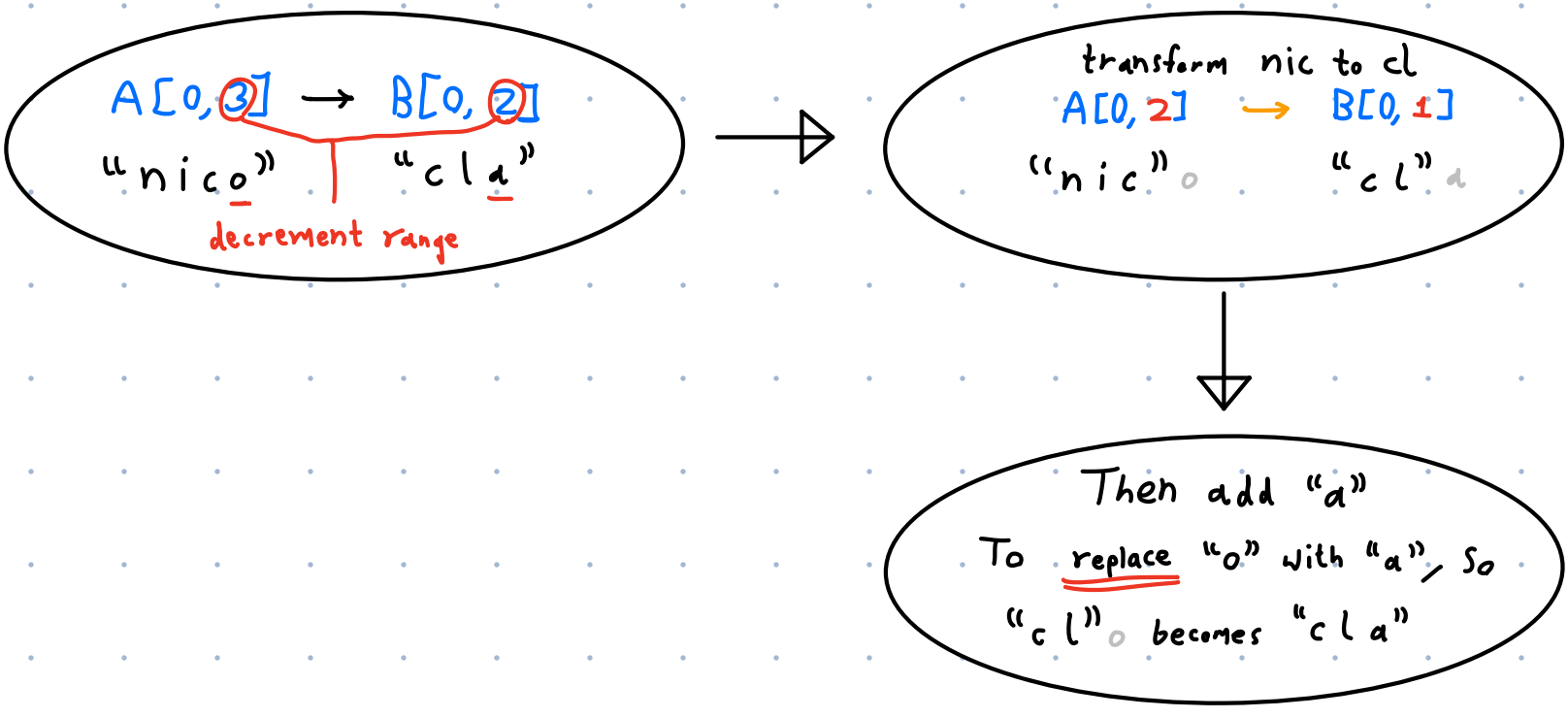
B = "claire"
0 1 2 3 4 5

Case 1: Match, Do nothing

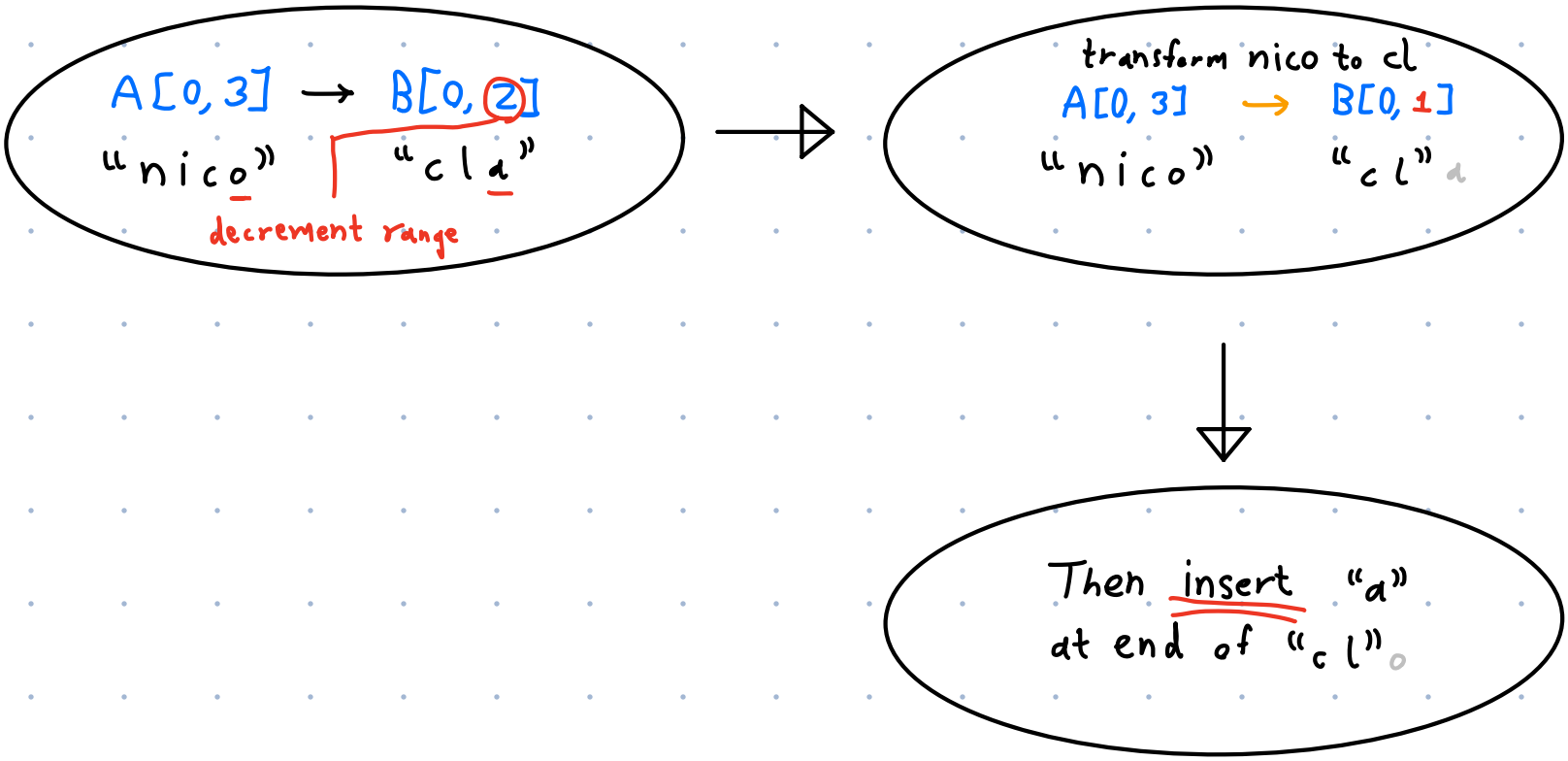
e is still there, we are just decreasing the range we are modifying



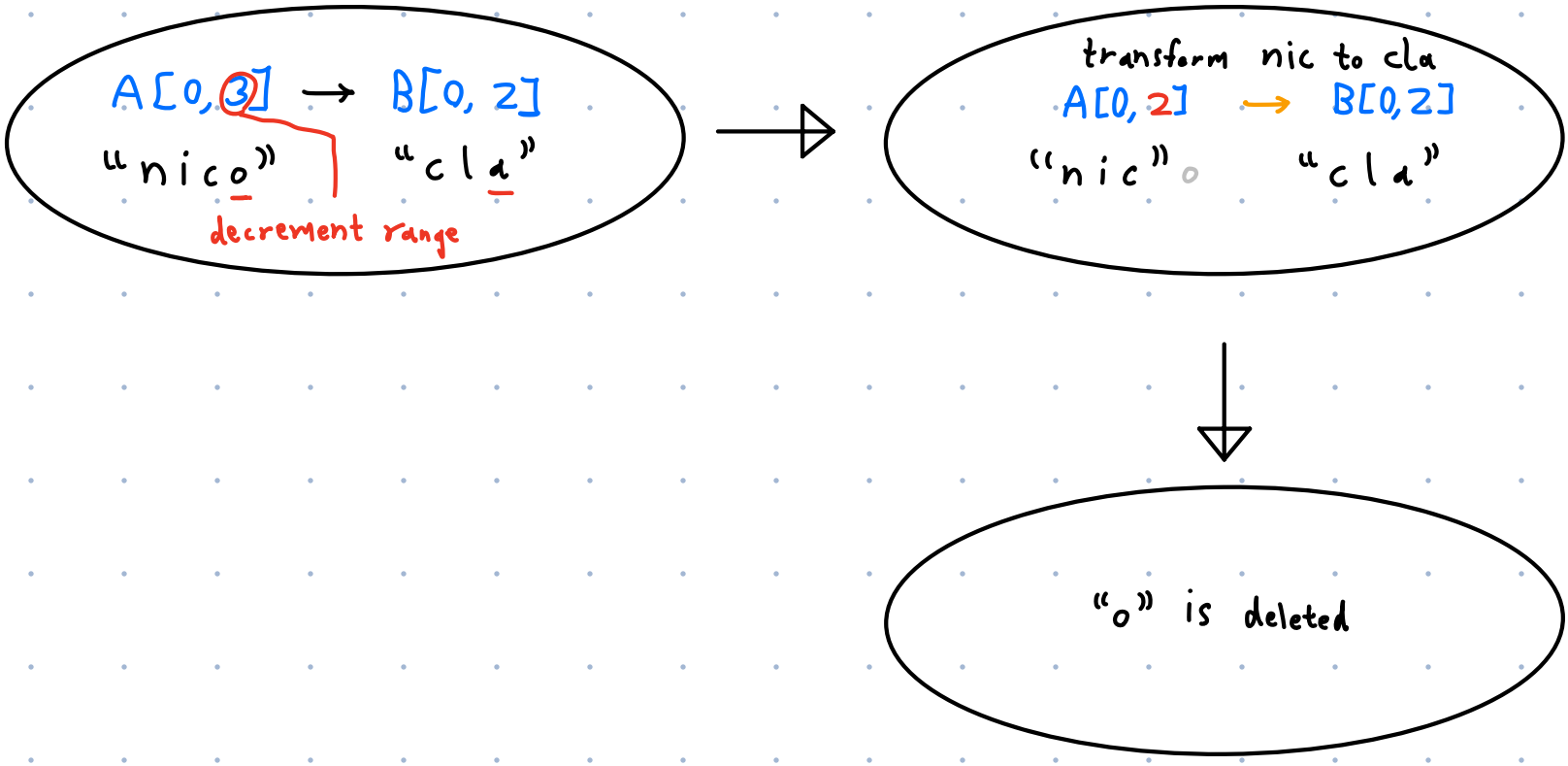
Case 2: Mismatch, Replace



Case 3: Mismatch, Insert



Case 4: Mismatch, Delete



The code part

```python
def levenshtein(a: str, b: str) -> float:
    """Return a similarity score (0 to 1) between two lines of text using the Levenshtein distance.

    Args:
        a: First line (old version)
        b: Second line (new version)

    Returns:
        A float between 0.0 and 1.0 where:
            1.0 means the lines match
            0.0 means the lines do not match
    """
    old_file_len, new_file_len = len(a), len(b)
```

6 6

		1 b	2 e	3 n	4 y	5 a	6 m
1	e						
2	p						
3	h						
4	r						
5	e						
6	m						

```python
    # Create the matrix of cells (rows = a prefix, columns = b prefix), initialized to 0
    dynamic_programming = [[0 for _ in range(new_file_len + 1)] for _ in range(old_file_len + 1)]
```

String 1

String 2

	""	b	e	n	y	a	m
""	0	0	0	0	0	0	0
e	0	0	0	0	0	0	0
p	0	0	0	0	0	0	0
h	0	0	0	0	0	0	0
r	0	0	0	0	0	0	0
e	0	0	0	0	0	0	0
m	0	0	0	0	0	0	0

```python
    # Initialize the base cases: cost of converting a prefix to/from the empty string
    for i in range(old_file_len + 1):
        dynamic_programming[i][0] = i   # i deletions to turn first i chars of a into empty
    for j in range(new_file_len + 1):
        dynamic_programming[0][j] = j   # j insertions to turn empty into first j chars of b
```

	""	b	e	n	y	a	m
""	0	1	2	3	4	5	6
e	1	0	0	0	0	0	0
p	2	0	0	0	0	0	0
h	3	0	0	0	0	0	0
r	4	0	0	0	0	0	0
e	5	0	0	0	0	0	0
m	6	0	0	0	0	0	0

```python
    # Fill the dynamic programming matrix
    for i in range(1, old_file_len + 1):
        for j in range(1, new_file_len + 1):
```

```python
            else:
                # Case 2: mismatch, a single edit needed
                cost = 1

            dynamic_programming[i][j] = min(
                dynamic_programming[i - 1][j - 1] + cost,     # Replace (or keep if cost = 0)
                dynamic_programming[i][j - 1] + 1,            # Insert
                dynamic_programming[i - 1][j] + 1             # Delete
            )
```

min:

0	1
1	0

0 + cost

0 + 1

= 1

j

i

	""	b	e	n	y	a	m
""	0	1	2	3	4	5	6
e	1	1	0	0	0	0	0
p	2	0	0	0	0	0	0
h	3	0	0	0	0	0	0
r	4	0	0	0	0	0	0
e	5	0	0	0	0	0	0
m	6	0	0	0	0	0	0

```python
# Fill the dynamic programming matrix
for i in range(1, old_file_len + 1):
    for j in range(1, new_file_len + 1):

        if string1[i - 1] == string2[j - 1]:
            # Case 1: match, do nothing
            cost = 0
        else:
            # Case 2: mismatch, a single edit needed
            cost = 1

        dynamic_programming[i][j] = min(
            dynamic_programming[i - 1][j - 1] + cost,     # Replace (or keep if cost = 0)
            dynamic_programming[i][j - 1] + 1,            # Insert
            dynamic_programming[i - 1][j] + 1             # Delete
        )
```

min: 1, 2, 1, 0

1 + cost

1 + 0

= 1

	""	b	e	n	y	a	m
""	0	1	2	3	4	5	6
e	1	1	1	0	0	0	0
p	2	0	0	0	0	0	0
h	3	0	0	0	0	0	0
r	4	0	0	0	0	0	0
e	5	0	0	0	0	0	0
m	6	0	0	0	0	0	0

```python
# Fill the dynamic programming matrix
for i in range(1, old_file_len + 1):
    for j in range(1, new_file_len + 1):

        else:
            # Case 2: mismatch, a single edit needed
            cost = 1

        dynamic_programming[i][j] = min(
            dynamic_programming[i - 1][j - 1] + cost,     # Replace (or keep if cost = 0)
            dynamic_programming[i][j - 1] + 1,            # Insert
            dynamic_programming[i - 1][j] + 1             # Delete
        )
```

min: 2, 3, 1, 0

1 + 1

= 2

	""	b	e	n	y	a	m
""	0	1	2	3	4	5	6
e	1	1	1	2	0	0	0
p	2	0	0	0	0	0	0
h	3	0	0	0	0	0	0
r	4	0	0	0	0	0	0
e	5	0	0	0	0	0	0
m	6	0	0	0	0	0	0

```python
# Fill the dynamic programming matrix
for i in range(1, old_file_len + 1):
    for j in range(1, new_file_len + 1):

        else:
            # Case 2: mismatch, a single edit needed
            cost = 1

        dynamic_programming[i][j] = min(
            dynamic_programming[i - 1][j - 1] + cost,     # Replace (or keep if cost = 0)
            dynamic_programming[i][j - 1] + 1,            # Insert
            dynamic_programming[i - 1][j] + 1             # Delete
        )
```

min: 5, 4, 6, 0

4 + 1

= 5

	""	b	e	n	y	a	m
""	0	1	2	3	4	5	6
e	1	1	1	2	3	4	5
p	2	2	2	2	3	4	5
h	3	3	3	3	3	4	5
r	4	4	4	4	4	4	5
e	5	5	4	5	5	5	5
m	6	6	5	0	0	0	0

```python
distance = dynamic_programming[old_file_len][new_file_len]
max_len = max(old_file_len, new_file_len)

if max_len == 0:    # Check if both string are empty
    return 1.0

return 1 - distance / max_len
```

	""	b	e	n	y	a	m
""	0	1	2	3	4	5	6
e	1	1	1	2	3	4	5
p	2	2	2	2	3	4	5
h	3	3	3	3	3	4	5
r	4	4	4	4	4	4	5
e	5	5	4	5	5	5	5
m	6	6	5	5	6	6	5

Full Code

```python
def levenshtein(a: str, b: str) -> float:
    """Return a similarity score (0 to 1) between two lines of text using the Levenshtein distance.

    Args:
        a: First line (old version)
        b: Second line (new version)

    Returns:
        A float between 0.0 and 1.0 where:
            1.0 means the lines match
            0.0 means the lines do not match
    """
    old_file_len, new_file_len = len(a), len(b)

    # Create the matrix of cells (rows = a prefix, columns = b prefix), initialized to 0
    dynamic_programming = [[0 for _ in range(new_file_len + 1)] for _ in range(old_file_len + 1)]

    # Initialize the base cases: cost of converting a prefix to/from the empty string
    for i in range(old_file_len + 1):
        dynamic_programming[i][0] = i   # i deletions to turn first i chars of a into empty
    for j in range(new_file_len + 1):
        dynamic_programming[0][j] = j   # j insertions to turn empty into first j chars of b

    # Fill the dynamic programming matrix
    for i in range(1, old_file_len + 1):
        for j in range(1, new_file_len + 1):
            if a[i - 1] == b[j - 1]:
                # Case 1: match, do nothing
                cost = 0
            else:
                # Case 2: mismatch, a single edit needed
                cost = 1

            dynamic_programming[i][j] = min(
                dynamic_programming[i - 1][j - 1] + cost,    # Replace (or keep if cost = 0)
                dynamic_programming[i][j - 1] + 1,           # Insert
                dynamic_programming[i - 1][j] + 1            # Delete
            )
    distance = dynamic_programming[old_file_len][new_file_len]
    max_len = max(old_file_len, new_file_len)

    if max_len == 0:    # Check if both string are empty
        return 1.0

    return 1 - distance / max_len
```